Discord Authenticator – Bericht

Joel Damm (jd106), Joshua Jung (jj050)

Inhalt

1. Projektziel und Problemstellung **2**
2. Anforderungen **3**
2.1 Muss-Ziele (funktionale Kernanforderungen) 3
2.2 Kann-Ziele (Qualitäts- und Architekturziele) 4
3. Systemarchitektur **5**
3.1 Gesamtarchitektur 5
3.2 Reverse Proxy (Caddy) 6
4. OAuth2-Authentifizierungsprozess **7**
Ablauf: 7
5. Schlüsselverwaltung und Verschlüsselung **9**
5.1 Token-Verschlüsselung 9
5.2 Schlüsselverwaltung 10
6. Erfüllung der Schutzziele (CIA-Modell) **12**
6.1 Vertraulichkeit 12
6.2 Integrität 13
6.3 Verfügbarkeit 13
7. Verwendete Technologien und Libraries **14**
Backend 14
Infrastruktur 14
8. Projektverlauf **15**
9. Bewertung und Fazit **16**

1. Projektziel und Problemstellung

Ziel dieses Projekts war die Entwicklung eines eigenständigen Authentifizierungsdienstes, der Nutzern ermöglicht, sich mittels ihres Discord-Accounts bei einer Webanwendung anzumelden.

Der Dienst implementiert den OAuth-2.0 Authorization Code Flow mit PKCE und orientiert sich an industriellen Sicherheitsstandards. Ziel war nicht nur eine funktionierende Anmeldung, sondern eine sicherheitsbewusste, skalierbare und erweiterbare Architektur.

Das System wurde als Full-Stack-Lösung konzipiert und umfasst:

- Backend (Rust, Axum)
- Frontend (SvelteKit)
- PostgreSQL (persistente Nutzerdaten)
- Redis (Session-Management)
- Reverse Proxy (Caddy mit TLS)
- Containerisierung mit Docker

Die Datei “README.md” im Git-Repository enthält die Anleitung zur Installation.

2. Anforderungen

2.1 Muss-Ziele (funktionale Kernanforderungen)

Diese Anforderungen waren zwingend notwendig:

- OAuth2 Authorization Code Flow
- PKCE-Absicherung
- CSRF-Schutz
- Sichere Session-Verwaltung
- HttpOnly- und Secure-Cookies
- TLS-gesicherte Kommunikation
- Persistente Speicherung von Nutzern
- Trennung von Code und Secrets

Ohne diese Anforderungen wäre das System nicht sicher betreibbar.

2.2 Kann-Ziele (Qualitäts- und Architekturziele)

Diese Ziele erhöhen Robustheit und Professionalität:

- AES-256-GCM-Verschlüsselung von Refresh-Tokens
- TTL-basierte Session-Verlängerung
- Compile-Time Query Validation mit SQLx
- Containerisierte Infrastruktur
- Logging via tracing
- Admin-Tools (pgAdmin, RedisInsight)
- Monorepo-Architektur

3. Systemarchitektur

3.1 Gesamtarchitektur

Das System ist in klar getrennte Komponenten aufgeteilt:

1. Reverse Proxy (Caddy)
2. Auth-Service (Rust)
3. PostgreSQL-Datenbank
4. Redis Session Store
5. Frontend (SvelteKit)

Die Services kommunizieren innerhalb eines isolierten Docker-Netzwerks.

Diese Trennung ermöglicht:

- Skalierbarkeit
- Wartbarkeit
- Fehlerisolation
- klare Sicherheitsgrenzen

3.2 Reverse Proxy (Caddy)

Der Reverse Proxy übernimmt:

- TLS-Terminierung
- HTTPS-Weiterleitung
- Zertifikatsmanagement (ACME)
- Weiterleitung interner Services

Sicherheitswirkung:

- Schutz vor Man-in-the-Middle-Angriffen
- Erzwingen verschlüsselter Verbindungen
- zentrale Sicherheitsgrenze (Security Boundary)

4. OAuth2-Authentifizierungsprozess

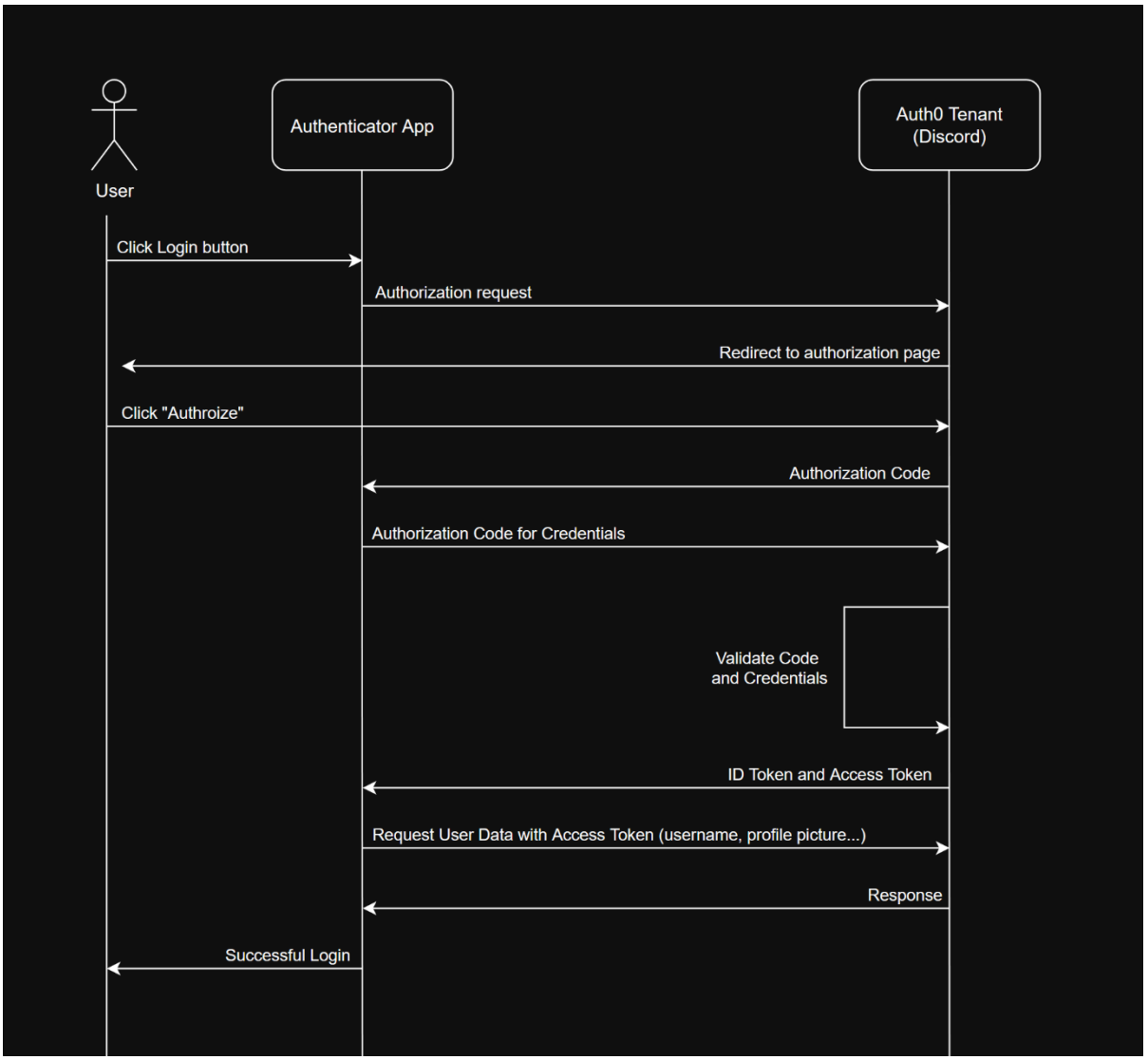
Der Authentifizierungsprozess basiert auf dem Authorization Code Flow mit PKCE.
Basierend auf: https://oauth.net/2/

Ablauf:

1. Erzeugung von PKCE-Challenge, CSRF-Token und Session-ID
2. Speicherung der temporären OAuth-Session in Redis
3. Weiterleitung zu Discord
4. Callback mit Authorization Code
5. Validierung von:
    - CSRF-Token
    - PKCE-Verifier
6. Austausch des Codes gegen Access- und Refresh-Token
7. Abruf von Benutzerdaten
8. Persistierung in PostgreSQL
9. Erzeugung einer serverseitigen Session in Redis
10. Übergabe eines HttpOnly-Session-Cookies an das Frontend

Es werden zu keinem Zeitpunkt Benutzerpasswörter verarbeitet oder gespeichert.

Abb. 1 OAuth2 flow, wie beschrieben auf: https://aaronparecki.com/oauth-2-simplified/



5. Schlüsselverwaltung und Verschlüsselung

5.1 Token-Verschlüsselung

Refresh-Tokens werden mit AES-256-GCM verschlüsselt gespeichert.

Verwendete Library: aes-gcm

Eigenschaften von AES-256-GCM:

- 256-Bit symmetrischer Schlüssel
- Authenticated Encryption (Integrität + Vertraulichkeit)
- 96-Bit Nonce
- Manipulationserkennung durch Authentication Tag

Bei Speicherung werden erzeugt:

- Ciphertext
- zufällige Nonce

Ohne korrekten Schlüssel ist eine Entschlüsselung kryptographisch nicht praktikabel.

5.2 Schlüsselverwaltung

Der Verschlüsselungsschlüssel:

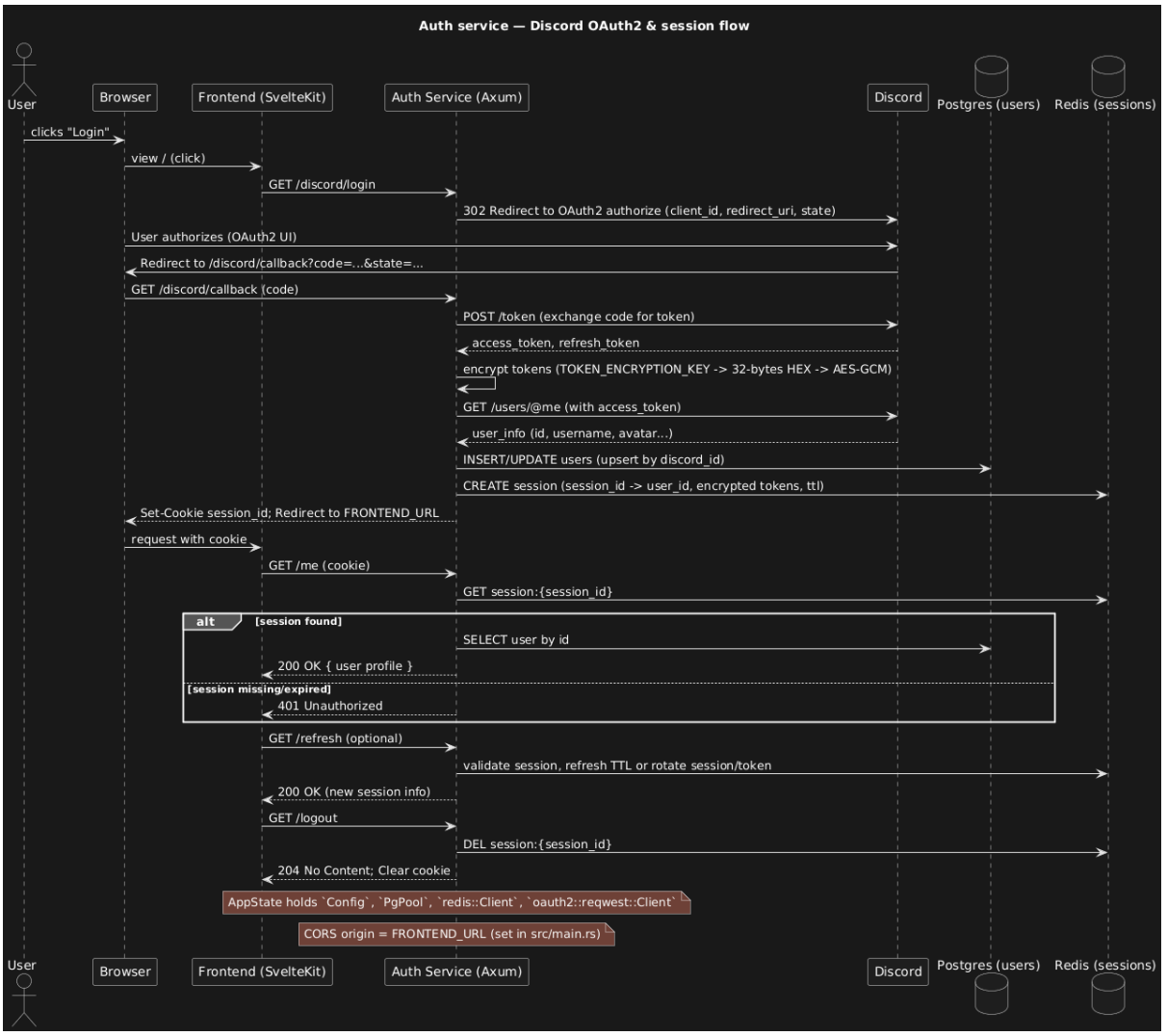
- wird als 32-Byte HEX-Wert gespeichert
- liegt in einer Environment-Variable (TOKEN_ENCRYPTION_KEY)
- wird über Docker Compose injiziert
- ist nicht im Repository enthalten

Damit wird erreicht:

- Trennung von Code und Secrets
- Austauschbarkeit des Schlüssels
- keine Hardcoded-Secrets

Für produktive Systeme wären zusätzlich Secret-Manager oder Docker-Secrets empfehlenswert.

Abb.2: Sequenzdiagramm des Auth-Service, dargestellt sind Anmeldung und Abmeldung.



6. Erfüllung der Schutzziele (CIA-Modell)

6.1 Vertraulichkeit

Erreicht durch:

- TLS-Verschlüsselung (Caddy)
- Secure-Cookies
- HttpOnly-Cookies
- AES-256-GCM für Tokens
- Environment-basierte Secret-Verwaltung

Schützt vor:

- Abhören von Daten
- Token-Diebstahl via XSS
- Klartextspeicherung sensibler Daten

6.2 Integrität

Erreicht durch:

- PKCE-Mechanismus
- CSRF-State-Validierung
- GCM-Authentifizierungstag
- Typensicherheit durch Rust
- Compile-Time SQL-Prüfung (SQLx)

Schützt vor:

- Code-Injection im OAuth-Flow
- manipulierten Tokens
- SQL-Injection
- unvalidierten Datenbankzugriffen

6.3 Verfügbarkeit

Erreicht durch:

- Containerisierte Services
- Redis für schnelle Session-Verwaltung
- Trennung von Persistenz und Session Layer
- isoliertes Docker-Netzwerk

Erhöht:

- Skalierbarkeit
- Wartbarkeit
- Fehlerisolierung

7. Verwendete Technologien und Libraries

Backend

- axum – Web Framework
- tokio – Async Runtime
- oauth2 – OAuth2-Implementierung
- sqlx – Compile-Time Query Validation
- redis – Async Session Store
- aes-gcm – Kryptographische Verschlüsselung
- serde / serde_json – Serialisierung
- uuid – Session-ID Generierung
- rand – kryptographische Nonce-Erzeugung
- tower-http – CORS Middleware
- tracing – Logging

Infrastruktur

- PostgreSQL
- Redis
- Docker & Docker Compose
- Caddy Reverse Proxy
- SvelteKit (Frontend)

8. Projektverlauf

Die Entwicklung begann mit dem Backend, da:

- User- und Session-Management die höchste Komplexität aufwiesen
- Rust und SQLx neu erlernt werden mussten

Herausforderungen:

- OAuth2-Integration
- Cookie-Weiterleitung hinter Reverse Proxy
- SQLx-Migrationsfehler
- Docker-Build-Zeiten
- Dependency-Konflikte

Durch iterative Verbesserung konnte ein stabiler Prototyp realisiert werden.

Was im Verlauf des Projekts gelernt wurde:

1. Korrektes Aufsetzen einer Entwicklungsumgebung kann viele Zeitaufwändige Probleme vermeiden
2. Build- und Compile- Zeiten dürfen nicht unterschätzt werden und müssen eingeplant werden
3. Containerisierung führt zu langen Build-Zeiten und sollte möglichst spät im Projekt implementiert werden
4. Komplexität kann schnell eskalieren und darf nicht unterschätzt werden
5. Vorausplanen und Visualisieren von Prozessen (z.B. mithilfe von Diagrammen) verschafft Überblick und ein klares Ziel

9. Bewertung und Fazit

Das Projekt verbindet:

- IT-Sicherheit
- Moderne Webarchitektur
- Containerisierung
- Datenbankdesign
- OAuth2-Standards

Zentrale Erkenntnisse:

1. Security muss von Beginn an Teil der Architektur sein.
2. Compile-Time Checks reduzieren signifikant Runtime-Fehler.
3. Trennung von Secrets und Code ist essenziell.
4. Industriestandards wie PKCE und AES-GCM sind unerlässlich für sichere Systeme.

Das System erfüllt die grundlegenden Schutzziele der Informationssicherheit (Vertraulichkeit, Integrität, Verfügbarkeit) und orientiert sich an industriellen Best Practices.

Es stellt eine sichere und skalierbare Grundlage dar und kann um weitere OAuth-Provider oder Frontends erweitert werden.